

Bauftrag: Update-Automatisierung ausbauen — drei Bausteine

Auslöser: Adam, 10.08.2026 05:30 Uhr, als Reaktion auf die wöchentliche 5.21-Update-Monitor-Meldung. **Für:** Mick (Umsetzung im Bot-Repo). **Zur Kenntnis:** Conni (Gegenlesung). **Diese Sitzung (Claudia) darf das Bot-Repo nicht schreiben** — reiner Vorschlag.

0. Was heute schon existiert (verifiziert, nicht neu zu bauen)

- `scripts/updater.py` + `/updates` -Befehl (`bot.py:3271` `cmd_updates`) + `upd: -Callback` (`bot.py:3329` `on_update_callback`): zeigt verfügbare Updates mit Ampel, bietet Freigabeknöpfe. Deterministisch, kein Modell-Aufruf.
- **Ampel existiert bereits** (`components.json` , `updater.classify()`): Patch/Minor, nicht gepinnt → sammelbar (ein Knopf „Alle sicheren einspielen“). gepinnt → nur Einzel-Freigabe. Major → Einzel-Freigabe + Rollback. `kind != "pip"` (npm/OS) → manuell, kein Knopf.
- **Pro Einzel-Update (/) zwei Wege** (`bot.py:3303-3311`): „jetzt“ sofort einspielen, oder „ 04:00 “ — fürs nächste automatische Wartungsfenster vormerken (`scripts/wartungsfenster.py`). Vorgemerkt ist stornierbar.
- **Sicherheitskette beim Einspielen** (gehärtet 25.07., Conni A1-A7 + C2, `scripts/test_updater_haertung.py` , 9 Prüfungen):
 - A1: vollständiger `pip freeze` **vor** jedem Update als Rollback-Grundlage.
 - A2: Rollback-Fehler laut melden, Zustand je Paket einzeln, nie fälschlich „erledigt“.
 - A3: installiert wird exakt die **angezeigte** Version (`==`), nicht `-U` — bei Drift zwischen Anzeige und Klick wird neu gefragt statt blind installiert.
 - A4: Lauf-Sperre (kein doppelter gleichzeitiger Lauf).
 - A5: Grundlinie zuerst — ist das Fundament schon rot, wird nichts angefasst.

- A6: gleiche Testzahl vor/nach Rollback ⇒ Ursache liegt nicht am Update.
- A7: Wiederhol-Schutz.
- Health-Check nach jedem Install:
`scripts/regressiontest.sh` — bei Rot automatischer Rollback.
- **Damit ist Adams Frage „hatten wir das Rollback nicht schon eingebaut?“ klar mit Ja beantwortet** — die Sicherung existiert, ist gehärtet und aktiv.

1. Befund vor dem Bauen: zwei mögliche Ampel-Logiken, nicht abgeglichen

`requirements.txt` (Zeilen 6/11/12) zeigt:

```

claude-agent-sdk==0.2.127    (exakt gepinnt)
pymupdf>=1.24                (Bereich, NICHT exakt)
anthropic>=0.50              (Bereich, NICHT exakt)

```

Nach der oben dokumentierten Regel („ = gepinnt“) müssten `pymupdf` und `anthropic` eigentlich sein — die wöchentliche Monitor-Meldung (`version_monitor.py`, Montag 04:20) zeigte sie aber als . Ob `version_monitor.py` (passive Wochenmeldung) und `updater.classify()` (interaktiver `/updates`-Befehl) dieselbe Klassifizierungs-Regel benutzen, ist **ungeprüft** — diese Sitzung hat `classify()` nicht gelesen. **Erster Schritt für Mick:** abgleichen, ob beide Wege dieselbe Logik teilen; falls nicht, vereinheitlichen oder den Unterschied dokumentieren. Bis das geklärt ist, gilt als verlässlich nur die Ampel, die der **interaktive** `/updates`-Befehl live zeigt — nicht die passive Wochenmeldung.

`claude-code-cli-global` ist laut `components.json`
`kind: "npm_global"`, nicht `pip` — fällt damit **nicht** in `cmd_updates` s Einzel-Button-Liste (die filtert auf `kind == "pip"`). Bleibt also so oder so ein manueller Root-Schritt (`npm update -g @anthropic-ai/claude-code`), unabhängig von diesem Bauauftrag.

`claude-agent-sdk` ist eindeutig: `"pinned": true`, Registernotiz „Update NUR im Wartungsfenster nach `scripts/regressiontest.sh`“ — keine Ambiguität, gehört auf **04:00**.

2. Baustein A — Buttons an die wöchentliche Monitor-Meldung hängen

Heute erscheinen die Freigabe-Knöpfe nur, wenn `/updates` aktiv aufgerufen wird; die automatische Wochenmeldung (die Adam gerade beantwortet hat) ist reiner Text ohne Knöpfe — das ist die Lücke hinter seinem „3 Buttons“-Wunsch.

Empfehlung: keine Parallel-Logik bauen, sondern dieselben `upd:` -Callbacks direkt an die Wochenmeldung hängen (dieselbe Tastatur wie `cmd_updates` erzeugt, nur als Absender die Monitor-Nachricht statt der `/updates` -Antwort).

Eine echte Lücke bleibt: Für ein frisches (noch nicht vorgemerkt) `/ -Update` gibt es aktuell nur „jetzt“ oder „ 04:00“ — kein sauberes „jetzt nicht, aber auch nicht vergessen“. Storno gibt es nur für bereits vorgemerkte Einträge. Vorschlag: dritter Knopf „ **jetzt nicht**“ je Einzel-Item, der das Update quittiert/protokolliert (taucht bei der nächsten Meldung wieder auf, wenn immer noch aktuell), ohne etwas zu installieren oder vorzumerken. Damit deckt sich das Verhalten inhaltlich mit Adams drei Optionen (alles einspielen / fürs Fenster vormerken / jetzt nicht), nur je Einzel-Update statt als ein Knopf-Trio für die ganze Nachricht — bewusst so belassen, weil ein Sammel-Knopf für dem „nur Einzel-Freigabe“-Prinzip widerspräche (siehe 0.).

3. Baustein B — Automatischer Check für die zwei „manual“-Registereinträge

`components.json` führt `whisper.cpp` und `lobe-chat` bereits mit `"note"` -Hinweis auf den manuellen Prüfweg — das ist ein **bekannter, dokumentierter Rückstand**, kein übersehener Punkt.

- **whisper.cpp:** `api.github.com/repos/ggml-org/whisper.cpp/releases/latest` (Felder `tag_name`, `published_at`). **Wichtig — heute selbst erlebt:** eine Abfrage der normalen Releases-Webseite (HTML zusammengefasst) lieferte ein falsches Datum („4. August 2024“ statt richtig 2026) — vermutlich hat das zusammenfassende Modell aus Trainingswissen „aufgefüllt“ statt die Seite wirklich zu lesen. Die direkte API-Abfrage lieferte das korrekte Datum. **Für den automatischen Check gilt: nur**

die API abfragen, nie eine Seite scrapen+zusammenfassen lassen.

- **lobe-chat:** die stabile Release-Schiene ist `api.github.com/repos/lobehub/lobe-chat/releases/latest` — **nicht** die „Desktop Canary“-Schiene (andere Produktlinie, mehrmals täglich, nicht für den Produktivbetrieb gedacht, hätte sonst ständig Fehlalarm erzeugt).
- **Grenze, die bleibt:** Beide API-Checks sagen nur, was **upstream** neu ist — nicht, welche Version **lokal läuft**. Für `whisper.cpp` geht das (Git-Commit lokal lesbar). Für den `lobe-chat`-Container bräuchte es `docker inspect` — diese Sitzung hat **keinen Docker-Socket-Zugriff** (geprüft: „permission denied“). Zwei Wege: (a) der Check läuft im Bot-Prozess/-Systemd-Kontext, der vermutlich passende Rechte hat, oder (b) diese Sitzung bekäme gezielt Docker-Leserecht. **(b) ist eine bewusste Rechte-Erweiterung und gehört vor jeder Umsetzung einzeln vor Adam** (Sicherheits-Grundregel), nicht automatisch mitgewährt.

Zur Einordnung, heute geprüft (nicht Teil des Bauauftrags, nur Stand): `whisper.cpp` lokal auf einem Commit vom 11.07., neueste Version v1.9.2 vom 04.08. — rund drei bis vier Wochen Rückstand, aber seit 25.07. nur noch Rückweg (`faster-whisper` ist seitdem die Vorgabe), also geringe Dringlichkeit. `lobe-chat` stabil zuletzt v2.2.13 (01.08.) — ob die laufende Version davon abweicht, ungeprüft (s. o.).

4. Baustein C — Claudia löst Updates nach Rücksprache selbst aus

Adams Wunsch: nicht mehr zwingend Adams eigener Knopf-Tap, sondern „nach Absprache mit mir“ soll die Assistentin auslösen können. Das berührt E3/AGB („kein Modell-Aufruf nötig, kein stiller Auto-Install“, bisher bewusst so gebaut, u. a. um automatisierte, Abo-authentifizierte Modellaktionen ohne unmittelbaren Menschen-Trigger zu vermeiden).

Zwei Wege, mit Empfehlung:

1. **Empfehlung — fester Text-Trigger:** ein deterministischer Handler in `bot.py` erkennt ein exaktes Schlüsselwort/Kommando in Adams Chat-Antwort (z. B. „/updates_ja“ oder ein

festes Wort) und reicht es 1:1 an dieselbe upd: -Logik weiter, die heute der Button auslöst. Die eigentliche Installations-Ausführung bleibt damit außerhalb jeden Modell-Ermessens — nur der Weg dahin wird bequemer (Chat-Antwort statt Knopf-Suche). Das Vier-Augen-Prinzip bleibt: Adam muss weiterhin selbst den auslösenden Text schicken.

2. **Alternative — echtes Modell-in-der-Schleife:** Claudia ruft direkt die Ausführungsfunktion auf, weil sie aus dem Gesprächsverlauf „versteht“, dass Adam zugestimmt hat. Spart einen kleinen Bauschritt, hebt aber genau die Trennung auf, die beim Bau der Updater-Härtung bewusst eingezogen wurde.

Adams Entscheid nötig — sicherheits-/AGB-relevant, keine Empfehlung wird ohne seine Zustimmung umgesetzt.

5. Offene Entscheide, gebündelt

1. **claude-agent-sdk jetzt:** übers bestehende -04:00-Wartungsfenster laufen lassen (Empfehlung, folgt der bestehenden Registernotiz) — oder auf einen anderen, bewusst gewählten Termin legen (z. B. erst nach dem 17.08., wenn der reguläre Wiedereinstieg beginnt)?
2. **Baustein C:** Text-Trigger (Empfehlung) oder direkte Modell-Ausführung?
3. **Docker-Leserecht für diese Sitzung:** gezielt gewähren (eng gefasst, nur lesend/ inspect) oder der lobe-chat-Versionscheck bleibt beim Bot-Prozess/bei Mick?
4. **Ampel-Abgleich (Punkt 1):** vor oder unabhängig von A/B/C zuerst klären, damit die Wochenmeldung verlässlich wird?

6. Was unverändert bleibt

Die bestehende Härtung (A1–A7, C2), das Vier-Augen-Prinzip, Freeze-vor-Install und Auto-Rollback bei Rot werden durch A/B/C nicht angetastet — alle drei Bausteine setzen darauf auf, ersetzen nichts.